

Module 4: Conditionals, Loops, and Error Handling

Conditional Execution with `only_if`

The `only_if` option lets you execute a command only when a condition is true. This is how you make playbooks adapt to what they discover.

```
commands:
  - type: shell
    cmd: nmap -p 80 $TARGET

  - type: regex
    cmd: (\d+)/tcp open http
    output:
      PORT: "$MATCH_0"

  # Only run nikto (scans for known vulnerabilities) if we actually found
  # port 80 open
  - type: shell
    cmd: nikto -host $TARGET -port $PORT
    only_if: $PORT == 80
```

Comparison Operators

Operator	Description	Example
<code>==</code>	Equal	<code>\$PORT == 80</code>
<code>!=</code>	Not equal	<code>\$STATUS != 0</code>
<code><, <=, >, >=</code>	Ordering	<code>\$COUNT > 1</code>
<code>is, is not</code>	Identity	<code>\$VAR is None</code>
<code>=~</code>	Regex matches	<code>\$OUTPUT =~ .*open.*</code>
<code>!~</code>	Regex does not match	<code>\$OUTPUT !~ .*closed.*</code>
<code>not</code>	Negation	<code>not \$FLAG</code>

Important: Type Awareness

All variables are stored as **strings**. This affects comparisons (see [Python reference on comparisons](#) and [value vs. identity](#)):

```
# CAUTION: $PORT is the string "80", not the integer 80
# String-to-integer comparison with == returns False in Python

# Use string literal on the right side for reliable comparisons:
```

```
only_if: $PORT == 80          # Works for numeric comparison
only_if: $FLAG == "True"      # Use quotes for boolean-like strings

# Safest approach (use regex matching):
only_if: $PORT =~ ^80$        # Regex avoids type issues entirely
```

Checking if a Variable is Set

```
# Execute only if USERPW has a value:
only_if: $USERPW

# Execute only if USERPW is not set:
only_if: not $USERPW

# Check for None:
only_if: $USERPW is not None
```

Error Handling

exit_on_error

By default, AttackMate stops the playbook if a command fails (returns non-zero). Set `exit_on_error: False` to continue:

```
commands:
  # This might fail, but we want to continue either way
  - type: shell
    cmd: nikto -host $TARGET
    exit_on_error: False

  # This will execute even if the previous command failed
  - type: shell
    cmd: nmap $TARGET
```

error_if and error_if_not

Trigger an error based on the command's output, using regex patterns:

```
commands:
  # Fail if the output contains "Connection refused"
  - type: shell
    cmd: curl http://$TARGET
    error_if: ".*Connection refused.*"

  # Fail if the output does NOT contain "200 OK"
```

```
- type: shell
cmd: curl -I http://$TARGET
error_if_not: ".*200 OK.*"
```

Retry Logic with `loop_if` / `loop_if_not`

Sometimes a command needs to be retried because a service might not be ready yet, or a network request might fail intermittently.

```
commands:
  # Retry the curl up to 5 times until the response contains "200 OK"
  - type: shell
    cmd: curl -I http://$TARGET
    loop_if_not: ".*200 OK.*"
    loop_count: 5
```

Option	Description
<code>loop_if</code>	Retry if pattern matches the output
<code>loop_if_not</code>	Retry if pattern does not match the output
<code>loop_count</code>	Maximum retries (default: 3)

Sleep Between Retries

Between each retry, AttackMate pauses for `loop_sleep` seconds. This is a **global** setting configured in the `cmd_config` section of a configuration file, **not** on the command itself. The default is **5 seconds**.

```
# config.yml
cmd_config:
  loop_sleep: 10
```

```
# Pass the config file when running a playbook
attackmate --config config.yml playbook.yml
```

The config file also supports `command_delay`, which adds a pause **before every command** in the playbook (default: 0). This is useful for slowing down playbooks during demonstrations, or for spacing out commands so that individual log artifacts remain distinguishable during analysis.

```
# config.yml
cmd_config:
```

```
loop_sleep: 5
command_delay: 2
```

The `loop` Command

For iterating over lists or ranges, use the `loop` command. This is different from `loop_if/loop_if_not` (which retry a *single* command).

Iterating Over a List

```
vars:
  PORTS:
    - 21
    - 22
    - 80
    - 443

commands:
  - type: loop
    cmd: "items(PORTS)"
    commands:
      - type: shell
        cmd: nmap -p $LOOP_ITEM $TARGET
      - type: debug
        cmd: "Scanned port $LOOP_ITEM"
```

The current element is available as `$LOOP_ITEM`.

Iterating Over a Range

```
commands:
  - type: loop
    cmd: "range(1, 5)"
    commands:
      - type: debug
        cmd: "Iteration $LOOP_INDEX"
```

The start value is **inclusive**, the end value is **exclusive** (just like Python's `range()`). So `range(1, 5)` iterates over 1, 2, 3, 4. The current index is available as `$LOOP_INDEX`.

Loop Until a Condition

The helper script `check_status.sh` (in `walkthroughs/`) randomly returns "done" or "pending", so the loop runs a different number of iterations each time:

```

commands:
- type: loop
  cmd: "until($STATUS == done)"
  commands:
    - type: shell
      cmd: ./check_status.sh
    - type: regex
      cmd: "(done|pending)"
      output:
        STATUS: "$MATCH_0"
    - type: debug
      cmd: "Status: $STATUS"

```

Breaking Out of a Loop

Use `break_if` to exit a loop early. The helper script `check_port.sh` (in [walkthroughs/](#)) simulates a port check: it prints "yes" for port 22 and "no" for everything else, so the loop will break at index 22:

```

commands:
- type: loop
  cmd: "range(0, 100)"
  break_if: $FOUND == yes
  commands:
    - type: shell
      cmd: ./check_port.sh $LOOP_INDEX
    - type: regex
      cmd: "(yes|no)"
      output:
        FOUND: "$MATCH_0"
    - type: debug
      cmd: "Port $LOOP_INDEX: $FOUND"

```

The `sleep` Command

Pause execution for a fixed or random duration:

```

commands:
# Sleep for 5 seconds
- type: sleep
  seconds: 5

# Sleep for a random duration between 2 and 10 seconds
- type: sleep
  seconds: 10
  min_sec: 2
  random: True

```

Combining It All

Here is a pattern that combines conditionals, error handling, and loops:

```
vars:
  TARGET: 192.168.1.100
  SERVICES:
    - 21
    - 22
    - 80

commands:
  # Scan each port in the list
  - type: loop
    cmd: "items(SERVICES)"
    commands:
      - type: shell
        cmd: nmap -p $LOOP_ITEM $TARGET
        exit_on_error: False

      - type: regex
        cmd: (closed|open)
        mode: search
        output:
          STATUS: "$MATCH_0"

      - type: debug
        cmd: "Port $LOOP_ITEM status: $STATUS"
```